

NVFP4 Quantization Accelerator Architecture Modeling Report

Hardware Design for AI Accelerators — Final Project

Technology Node: 45 nm CMOS, 0.9 V

Modeling Framework: AccelForge (Timeloop-based)

Energy Reference: Horowitz, ISSCC 2014

Area Reference: TSMC 45 nm 6T SRAM, Lab3 die-photo estimates

1. Introduction

This report describes the architecture modeling methodology for an accelerator optimized for the NVIDIA FP4 (NVFP4) quantization pipeline. The NVFP4 format uses two-level scaling (per-tensor + per-block) to quantize FP16 activations and weights down to 4-bit floating point (E2M1), perform FP4 x FP4 matrix multiplication with FP32 accumulation, and then rescale the output back to FP16. The full pipeline consists of 13 einsum operations modeled explicitly in AccelForge.

The architecture features a three-level memory hierarchy (DRAM, GLB, RF), a 2D spatial PE array ($8 \times 8 = 64$ PEs), and three specialized compute units (QuantMAC, FP4MAC, RescaleMAC) with energy and area values derived from Horowitz's ISSCC 2014 reference data at the 45 nm node.

2. Workload Specification

2.1 Iteration Space

The workload operates over a four-dimensional iteration space that results from splitting the original K dimension into block (kb) and intra-block (ki) components, where $ki = 16$ is the NVFP4 block size:

Dimension	Range	Description
m	$0 \leq m < 64$	Activation row (output row)
n	$0 \leq n < 64$	Weight column (output column)
kb	$0 \leq kb < 8$	Block index along K
ki	$0 \leq ki < 16$	Intra-block index (NVFP4 block size)

Table 1: Iteration space dimensions.

2.2 Tensor Precision

The NVFP4 pipeline uses mixed precision across 15 tensors. The key insight is that FP4 (4-bit) is used only for the quantized operands (Aq, Wq) of the core matrix multiply, while FP32 is used for accumulation and rescaling:

Tensor	Bits	Shape	Role
A	16	[m, kb, ki]	FP16 input activations
W	16	[n, kb, ki]	FP16 input weights
Sga	32	[m]	Per-row activation scale (FP32)
Sgw	32	[n]	Per-column weight scale (FP32)
Ascl	16	[m, kb, ki]	Tensor-scaled activations
Wscl	16	[n, kb, ki]	Tensor-scaled weights
Sba	8	[m, kb]	Block scale activations (FP8 E4M3)
Sbw	8	[n, kb]	Block scale weights (FP8 E4M3)
Aq	4	[m, kb, ki]	Quantized activations (FP4 E2M1)

Tensor	Bits	Shape	Role
Wq	4	[n, kb, ki]	Quantized weights (FP4 E2M1)
Yraw	32	[m, n, kb]	FP32 partial sums
Ytmp	32	[m, n, kb]	After block-A rescale
Yblk	32	[m, n, kb]	After block-W rescale
Ytmp2	32	[m, n]	After tensor-A rescale
Y	16	[m, n]	Final FP16 output

Table 2: Tensor specifications. Total data footprint: 481.5 KB.

2.3 Einsum Pipeline (13 Operations)

The NVFP4 quantization pipeline is decomposed into 13 explicit einsum operations organized in four stages: activation quantization (4 ops), weight quantization (4 ops), core matrix multiply (1 op), and output rescaling (4 ops).

#	Einsum	Operation	Compute	Ops
1	TensorScaleA	$Sga[m] = \text{reduce}(A[m, kb, ki])$	QuantMAC	8,192
2	TensorQuantA	$Ascl = A / Sga$	QuantMAC	8,192
3	BlockScaleA	$Sba[m, kb] = \text{reduce}(Ascl) \text{ over } ki$	QuantMAC	8,192
4	BlockQuantA	$Aq = Ascl / Sba$	QuantMAC	8,192
5	TensorScaleW	$Sgw[n] = \text{reduce}(W[n, kb, ki])$	QuantMAC	8,192
6	TensorQuantW	$Wscl = W / Sgw$	QuantMAC	8,192
7	BlockScaleW	$Sbw[n, kb] = \text{reduce}(Wscl) \text{ over } ki$	QuantMAC	8,192
8	BlockQuantW	$Wq = Wscl / Sbw$	QuantMAC	8,192
9	MatMulNVFP4	$Yraw[m, n, kb] += Aq * Wq \text{ (reduce } ki)$	FP4MAC	524,288
10	RescaleBlockA	$Ytmp = Yraw * Sba$	RescaleMAC	32,768
11	RescaleBlockW	$Yblk = Ytmp * Sbw$	RescaleMAC	32,768
12	RescaleTensorA	$Ytmp2[m, n] = \text{sum}(Yblk * Sga) \text{ over } kb$	RescaleMAC	32,768
13	RescaleTensorW	$Y = Ytmp2 * Sgw$	RescaleMAC	4,096

Table 3: Einsum pipeline. Total: 692,224 ops. MatMulNVFP4 = 75.7%.

3. Architecture Design

3.1 Overview

The accelerator uses a three-level memory hierarchy with a 2D spatial array and three specialized compute units. The architecture is parameterized using Jinja2 templates, allowing design-space exploration of buffer sizes, PE counts, and bandwidth.

DRAM (off-chip, 64-bit bus) --> GLB (64 KB on-chip SRAM) --> RF (128 B/PE, 8x8 = 64 PEs) --> QuantMAC / FP4MAC / RescaleMAC

3.2 Memory Hierarchy

The three-level memory hierarchy is the most critical design decision. In the 13-einsum pipeline, intermediate tensors (Sga, Ascl, Sba, Aq, Wscl, Sbw, Wq, Yraw, Ytmp, Yblk, Ytmp2) are produced by one einsum and consumed by the next. A sufficiently large GLB allows the auto-mapper to fuse consecutive einsums, keeping intermediates on-chip and avoiding costly DRAM round-trips. Analysis of the baseline design showed that 88% of total energy was spent on memory access, making this the dominant optimization target.

Level	Name	Size	Energy/Action	Bus Width	BW (act/cyc)	Area
L0	DRAM	Unlimited	10.0 pJ	64-bit	8	0 (off-chip)
L1	GLB	64 KB	5.0 pJ	32-bit	32	0.39 mm ²
L2	RF	128 B / PE	1.0 pJ	32-bit	2	1024 um ² / PE

Table 4: Memory hierarchy. RF is replicated 64x (8x8 spatial array).

Tensor placement policy: DRAM stores non-intermediate tensors (A, W, Y) via keep: ~Intermediates. GLB stores all intermediates via keep: ~DRAM. RF is flexible (may_keep: All) and the auto-mapper decides what to cache per-PE for maximum reuse.

3.3 Spatial Array (2D, 8 x 8 = 64 PEs)

The register file level carries a 2D spatial attribute with fanout 8 in both X and Y dimensions, creating a grid of 64 processing elements. For the dominant MatMulNVFP4 einsum (Aq[m,kb,ki] x Wq[n,kb,ki] -> Yraw[m,n,kb]):

- X dimension maps to m: each column of PEs processes a different activation row.
- Y dimension maps to n: each row of PEs processes a different weight column.
- Aq is multicast along Y (shared across n); Wq is multicast along X (shared across m).
- This implements an **output-stationary dataflow**: each PE accumulates its own Yraw[m,n].

For quantization einsums that lack the n dimension (e.g., TensorScaleA operates on [m, kb, ki] only), the Y-axis PEs are underutilized. However, since quantization accounts for only 9.5% of total compute, this inefficiency is negligible.

3.4 Specialized Compute Units

Each PE contains three specialized compute units reflecting the heterogeneous precision requirements of the NVFP4 pipeline. Energy and area values are derived from Horowitz's 45 nm reference data, with

multiplier area scaling as the square of the mantissa bit-width:

Unit	Handles	Energy	Latency	Area/PE	Horowitz Basis
QuantMAC	8 quant einsums (9.5% compute)	1.0 pJ	1 cycle	90 um ²	16b FP mult ~ 1.0 pJ
FP4MAC	MatMulNVFP4 (75.7% compute)	0.2 pJ	1 cycle	60 um ²	8b Int mult = 0.2 pJ
RescaleMAC	4 rescale einsums (14.8% compute)	3.7 pJ	2 cycles	720 um ²	32b FP mult = 3.7 pJ

Table 5: Compute unit specifications per PE.

Key observation: FP4MAC (0.2 pJ) is 18.5x cheaper than RescaleMAC (3.7 pJ) per operation. Although MatMulNVFP4 dominates in operation count (75.7%), the FP32 rescaling steps contribute disproportionately to compute energy due to their high per-operation cost. This motivates hardware optimization of the rescale datapath in future designs.

4. Energy and Area Reference Values

4.1 Compute Energy (Horowitz, ISSCC 2014)

All compute energy values are derived from Horowitz's widely-cited energy table for 45 nm CMOS at 0.9 V [1]. The following reference values were used:

Operation	Energy (pJ)	Used For
8-bit Int Add	0.03	(reference)
16-bit Int Add	0.05	(reference)
32-bit Int Add	0.1	(reference)
8-bit Int Multiply	0.2	FP4MAC (FP4 ~ 8b int complexity)
32-bit Int Multiply	3.1	(reference)
32-bit FP Add	0.9	(reference)
16-bit FP Multiply	~1.0	QuantMAC (interpolated)
32-bit FP Multiply	3.7	RescaleMAC

Table 6: Horowitz 2014 energy reference (45 nm, 0.9 V).

4.2 Memory Energy

Memory	Horowitz Value	Our Model	Notes
8 KB SRAM (32b read)	5 pJ	(reference)	Closest to RF size
32 KB SRAM (32b read)	10 pJ	(reference)	Between GLB sizes
64 KB SRAM (32b read)	~10-13 pJ	5 pJ (GLB)	Scratchpad: no tag compare
Register file (32b)	~0.5-1 pJ	1 pJ (RF)	Small, near-compute
DRAM (64b access)	~1300 pJ	10 pJ	Amortized burst model

Table 7: Memory energy comparison. DRAM uses a simplified model consistent with the AccelForge lab framework, not the full Horowitz random-access cost.

4.3 Area Estimation

SRAM area is based on the TSMC 45 nm 6T cell size of $0.296 \text{ } \mu\text{m}^2/\text{bit}$ [2], with a 2.5x overhead factor for peripheral circuits (decoders, sense amplifiers, I/O). Compute unit areas use lab3's die-photo estimate of $90 \text{ } \mu\text{m}^2$ for a 16-bit MAC as the baseline, scaled by mantissa-width squared for other precisions [3].

Component	Formula	Per-Instance	Total (64 PEs)
GLB (64 KB)	bits x $0.296 \text{ } \mu\text{m}^2$ x 2.5	0.39 mm ²	0.39 mm ²
RF (128 B/PE)	bits x $1.0 \text{ } \mu\text{m}^2$	1,024 μm^2	0.066 mm ²
QuantMAC	FP16 MAC baseline	90 μm^2	0.006 mm ²
FP4MAC	FP4 mult + FP32 accum	60 μm^2	0.004 mm ²

Component	Formula	Per-Instance	Total (64 PEs)
RescaleMAC	8x FP16 (mantissa2)	720 um2	0.046 mm2
Total			0.51 mm2

Table 8: Area breakdown at 45 nm.

5. Design-Space Exploration Parameters

The architecture is implemented as a Jinja2 template (arch_template.yaml), allowing rapid exploration of the design space by adjusting parameters in the notebook without modifying the architecture file:

Parameter	Default	Unit	Affects
GLB_SIZE	524,288	bits	GLB capacity and area
RF_SIZE	1,024	bits	Per-PE RF capacity and area
PE_X	8	count	Spatial X fanout (array width)
PE_Y	8	count	Spatial Y fanout (array height)
DRAM_BW	8	actions/cyc	DRAM bandwidth (latency divisor)
GLB_BW	32	actions/cyc	GLB bandwidth (latency divisor)

Table 9: Parameterized architecture variables.

Area is automatically computed from parameters: GLB area = $\text{GLB_SIZE} \times 7.4\text{e-}13 \text{ m}^2$, RF area per PE = $\text{RF_SIZE} \times 1\text{e-}12 \text{ m}^2$. All parameters are passed to AccelForge via `jinja_parse_data` in `Spec.from_yaml()`.

6. Compute and Energy Analysis

6.1 Operation Count Breakdown

Stage	Einsums	Ops	% Total	Compute Unit	Energy/Op
Activation Quant	1-4	32,768	4.7%	QuantMAC	1.0 pJ
Weight Quant	5-8	32,768	4.7%	QuantMAC	1.0 pJ
MatMul (FP4xFP4)	9	524,288	75.7%	FP4MAC	0.2 pJ
Rescale	10-13	102,400	14.8%	RescaleMAC	3.7 pJ
Total	1-13	692,224	100%		

Table 10: Compute distribution by pipeline stage.

6.2 Compute Energy by Stage

Despite MatMulINVFP4 dominating operation count (75.7%), the per-operation energy differences create a more nuanced energy distribution:

Stage	Ops	Energy/Op (pJ)	Stage Energy (pJ)	% Compute Energy
Quant (A+W)	65,536	1.0	65,536	13.6%
MatMul (FP4xFP4)	524,288	0.2	104,858	21.8%
Rescale	102,400	3.7	378,880	78.8%
Total	692,224		549,274	

Table 11: Compute energy breakdown. Note: rescale (only 14.8% of ops) dominates compute energy at 78.8% due to expensive FP32 multiplications (latency: 2 cycles).

7. Data Flow and Fusion Strategy

The 13-einsum pipeline forms a directed acyclic graph (DAG) of tensor dependencies. The auto-mapper (AccelForge's FFM mapper) searches for optimal mappings that fuse consecutive einsums, keeping intermediate tensors in GLB:

```
A[DRAM] -> TensorScaleA -> Sga[GLB] -> TensorQuanta -> Asc1[GLB] ->
      BlockScaleA -> Sba[GLB] -> BlockQuanta -> Aq[GLB]

W[DRAM] -> TensorScaleW -> Sgw[GLB] -> TensorQuantW -> Wsc1[GLB] ->
      BlockScaleW -> Sbw[GLB] -> BlockQuantW -> Wq[GLB]

      Aq[GLB] + Wq[GLB] -> MatMulNVFP4 -> Yraw[GLB]

Yraw -> RescaleBlockA -> Ytmp -> RescaleBlockW -> Yblk -> RescaleTensorA ->
      Ytmp2 -> RescaleTensorW -> Y[DRAM]
```

Key insight: Only A, W are read from DRAM and Y is written back. All 12 intermediate tensors remain in GLB through fusion. This eliminates the dominant energy cost identified in the baseline analysis (88% of energy was memory access).

The auto-mapper uses AccelForge's `map_workload_to_arch()` with `Metrics.LATENCY` | `Metrics.ENERGY` to explore the mapping space. The best mapping is selected by minimum energy-delay product (EDP). The generated mapping is exported as YAML for reproducibility.

8. Design Rationale Summary

Design Choice	Rationale
3-level memory (DRAM/GLB/RF)	GLB enables fusion of 13-einsum pipeline; RF provides per-PE accumulator and tile storage
64 KB GLB	Holds all intermediates (~481 KB total, tiled) without DRAM spilling
2D spatial (8x8)	Output-stationary dataflow for MatMulNVFP4; both Aq and Wq get spatial reuse
3 compute units	Accurate energy modeling for FP4/FP16/FP32; reflects real HW heterogeneity (e.g., Blackwell)
64-bit DRAM bus	Halves DRAM action count vs 32-bit; matches FP32 accumulator width
RescaleMAC latency=2	FP32 multiply has ~2x critical path vs FP16; realistic for non-pipelined 45 nm design

Table 12: Summary of architecture design decisions.

References

- [1] M. Horowitz, "1.1 Computing's Energy Problem (and What We Can Do About It)," IEEE International Solid-State Circuits Conference (ISSCC), 2014.
- [2] TSMC, "45nm Node Planar-SOI Technology with 0.296 μm^2 6T-SRAM Cell," TSMC Research, 2008.
- [3] Google, "BFloat16: The Secret to High Performance on Cloud TPUs," Google Cloud Blog, 2019. (multiplier area $\sim$ mantissa_bits²)
- [4] NVIDIA, "Introducing NVFP4 for Efficient and Accurate Low-Precision Inference," NVIDIA Technical Blog, 2025.
- [5] A. Parashar et al., "Timeloop: A Systematic Approach to DNN Accelerator Evaluation," IEEE ISPASS, 2019.